IMPORTING REQUIRED LIBRARIES

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import classification_report
import seaborn as sn
import matplotlib.pyplot as plt
from sklearn import tree
```

IMPORTING (READING) DATASET

In [2]:
```
data=pd.read_csv('C:/Users/BALLA/Desktop/SERGE/JUPYTER/Breast Cancer Detection
print(data.shape)
data.head()
```

```
(569, 32)
```

Out[2]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compa
0	842302	17.99	10.38	122.80	1001.0	0.11840	
1	842517	20.57	17.77	132.90	1326.0	0.08474	
2	84300903	19.69	21.25	130.00	1203.0	0.10960	
3	84348301	11.42	20.38	77.58	386.1	0.14250	
4	84358402	20.29	14.34	135.10	1297.0	0.10030	

5 rows × 32 columns

In [3]:
```
data.sum()
```

Out[3]:
```
id                         1.728157e+10
radius_mean                8.038429e+03
texture_mean               1.097581e+04
perimeter_mean             5.233038e+04
area_mean                  3.726319e+05
smoothness_mean            5.482900e+01
compactness_mean           5.937002e+01
concavity_mean             5.052681e+01
concave points_mean        2.783499e+01
symmetry_mean              1.030811e+02
fractal_dimension_mean     3.573184e+01
radius_se                  2.305429e+02
texture_se                 6.923896e+02
perimeter_se               1.630788e+03
area_se                    2.295180e+04
smoothness_se              4.006317e+00
compactness_se             1.449706e+01
concavity_se               1.814752e+01
concave points_se          6.712002e+00
symmetry_se                1.168857e+01
fractal_dimension_se       2.159300e+00
radius_worst               9.257169e+03
texture_worst              1.461034e+04
perimeter_worst            6.103163e+04
area_worst                 5.010518e+05
smoothness_worst           7.531773e+01
compactness_worst          1.446768e+02
concavity_worst            1.548752e+02
concave points_worst       6.521094e+01
symmetry_worst             1.650530e+02
fractal_dimension_worst    4.776517e+01
diagnosis                  2.120000e+02
dtype: float64
```

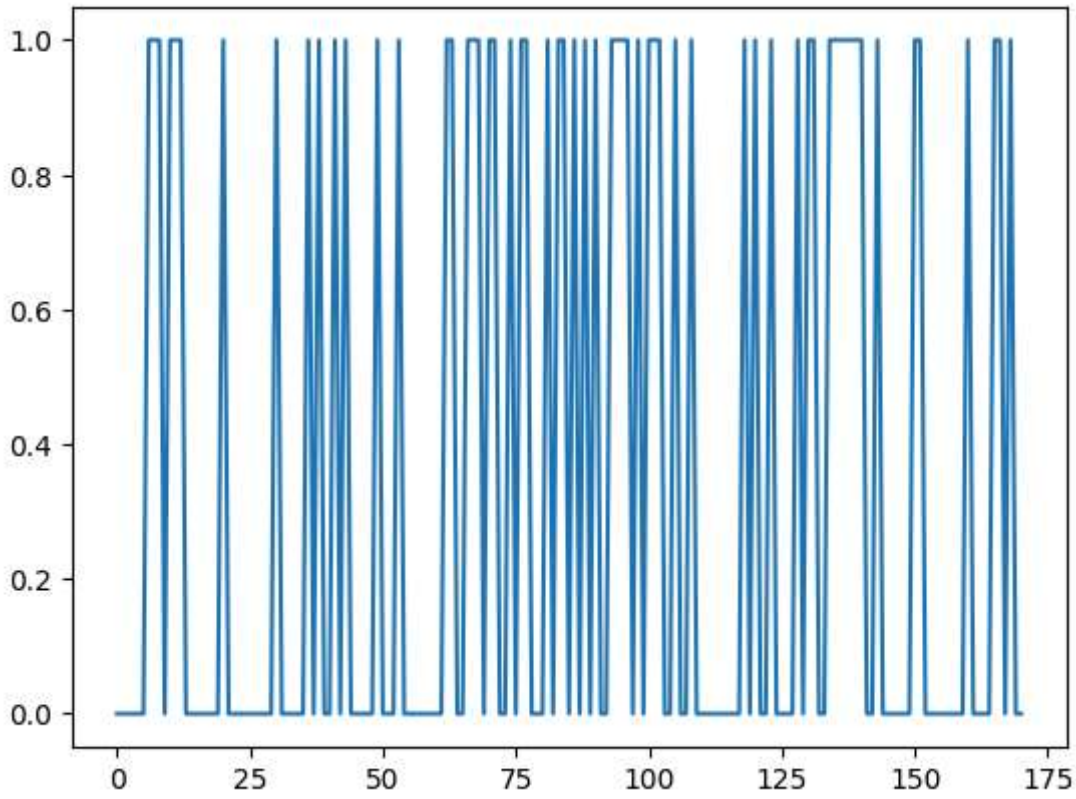
In [8]:
```
#Assigning dependent and independent variables
x=data.iloc[:,:-1].values
y=data.iloc[:,-1].values
```

In [9]:
```
#Splitting the dataset into Training and Testing Dataset
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=
```

In [10]:
```
#Preprocessing Data with StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [11]:
```
model=RandomForestClassifier(n_estimators=10,criterion='entropy',random_state=
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
y_pred
plt.plot(y_pred)
```

Out[11]:
```
[<matplotlib.lines.Line2D at 0x1e541e5d8d0>]
```



In [12]:
```
print('Random Forest Classifier')
conf_mat=metrics.confusion_matrix(y_test, y_pred)
print('\n Confusion Matrix : \n', conf_mat)
Accuracy_score=accuracy_score(y_test,y_pred)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ',
int(Accuracy_score*100),'%')
print('\n',classification_report(y_pred,y_test))
```

```
Random Forest Classifier

 Confusion Matrix :
 [[110   0]
 [  1  60]]
Accuracy Score :  0.9941520467836257
Accuracy in Percentage :  99 %

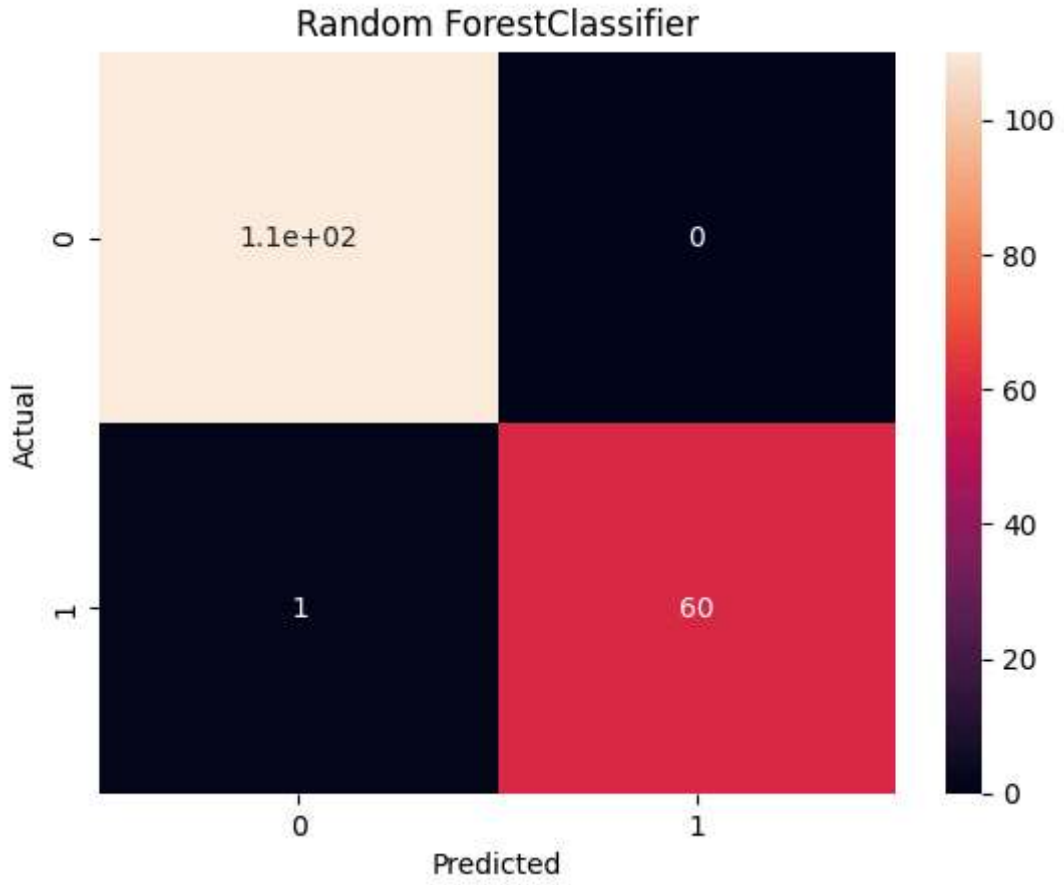
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       111
           1       0.98      1.00      0.99        60

    accuracy                           0.99       171
   macro avg       0.99      1.00      0.99       171
weighted avg       0.99      0.99      0.99       171
```

In [13]:
```
conf_mat=pd.crosstab(y_test,y_pred, rownames=['Actual'],colnames=['Predicted'])
sn.heatmap(conf_mat, annot=True).set(title='Random ForestClassifier')
```

Out[13]: [Text(0.5, 1.0, 'Random ForestClassifier')]



In [ ]:

In [ ]:

In [ ]: